CISC 221

Assignment 3

Arham Hassan | 20426387

Machine Code | Part A:

Q1) (3.60)

```
long loop(long x, int n)
{
    long result = 0 ;
    long mask;
    for (mask = 1 ; mask != 0 ; mask = mask << n) {
        result |= (x & mask ;
    }
    return result;
}
```

Q2) (3.64)

A) Formula: & A[i] [j] [k] = x_A + 8 * ((i*S + j)*T + k)
B) R = 7, S = 5, T = 13

Q3) (3.67)

Part A:

0x00 : s.a[0] = x
0x08 : s.a[1] = y
0x10 : s.p = &z
0x18 : z
0x20..0x3F :
0x40 : r.u[0]
0x48 : r.u[1]
0x50 : r.q
0x58..0x67 :

So, we have a total of 104 bytes reserved, and the extra padding should keep us aligned for the call.

Part B:

- Eval passes %rdi = &r, meaning, the address at 64(%rsp) and the space in the caller's frame where the result structure must be written. The argument structure s is passed by the value in the memory: its 24 bytes are already placed at (%rsp), 8(%rsp), 16(%rsp) for the callee to read. On top of that no extra register is used for s.

Part C: At the entry to process, the caller's %rsp was decremented by the call, so s is found at **8(%rsp)..24(%rsp)** which is relative to the callee, as we can see in lines 3-7.

We can conclude that: s.p is at 24, s.a[1] at 16 and s.a[0] at 8.

Part D: The sret pointer which is hidden at the destination for strB is in %rdi. The callee register writes the fields by the fixed offsets, and then it returns the same pointer in %rax. This shows that the structure fields compile
As constant offsets, which is the standard struct layout.

Part E:
This process gives us the result into the area starting at 64(%rsp). eval then sums:

```
10 movq 72(%rsp), %rax    ; load r.u[1]
11 addq 64(%rsp), %rax    ; + r.u[0]
12 addq 80(%rsp), %rax    ; + r.q
```

So the mapping technically is:

64(%rsp) : r.u[0]
72(%rsp) : r.u[1]
80(%rsp) : r.q
And then it should deallocate the frame (addq $104, %rsp) and then return it.

Part F:

- Struct argument is being passed by the value. So, caller will copy the bytes of the struct to memory (typically it is the stack frame); callee reads fields via fixed offsets from %rsp, or from a pointer.

- Struct results being returned by the value (size > 16 B), meaning, caller will allocate the space and pass a hidden destination pointer in %rdi. After which the callee stores into that memory and returns the same pointer in %rax. (as we can see in the leaq 64(%rsp), %rdi in caller and movq %rdi,%rax in callee.)

- Visible Register usage: integer/pointer args will go in %rdi,%rsi,%rdx,%rcx,%r8,%r9 when it's used, otherwise, memory is used (as here for s). And the return value is in %rax.

Q4) **(3.69)**

Part A) Find CNT:

We can see that:

- last is read from `0x120(%rsi)`. Meaning, offset of `last` in `b_struct` = 0x120 = 288 bytes.
- first is at `(%rsi)`. Meaning, offset is 0 (for 32-bit load).

For address of a[i]:

- lea (%rdi,%rdi,4),%rax → rax = 5*i.
- lea (%rsi,%rax,8),%rax → rax = bp + 8*(5*i) = bp + 40*i.

So, sizeof(a_struct) = 40 (bytes).

- In b_struct, `a` starts after `first` with 4 bytes padding for 8-byte alignment, meaning, offset 8.

Hence our last offset would be = 8 + CNT * 40 = 288, which gives us: CNT = (288-8)/40 = 7.

So, our Answer is: CNT = 7.

Part B):

- mov 0x8(%rax), %rdx right after rax = bp + 40*i.

Since &a[i] = bp + 8 + 40*i, 0x8(%rax) = &a[i], this should load `ap->idx`.
It's a 64-bit load → `idx` is `long` (signed).

- Store: mov %rcx, 0x10(%rax,%rdx,8).

Address = `bp + 40*i + 0x10 + idx*8` = `(&a[i]) + 0x8 + idx*8`.
So the base of `x` is 8 bytes into `a_struct`, and the elements are 8-byte each give that, `x` is an array of `long`.

- Size cross-check: If `idx` is `long` at offset 0 and `x` starts at offset 8, with `sizeof(a_struct)=40`, then `x` occupies `40−8 = 32` bytes ⇒ 4 elements.

Hence, answer:

```c
typedef struct {
  long idx;      // offset 0, 8 bytes (signed)
  long x[4];     // offset 8, 32 bytes total (4 signed long elements)
} a_struct;        // sizeof = 40 bytes
```